

ACI · AIMLCZG557

AI Agents, PEAS & Environments

Advanced Computing for AI — Lesson 1

Exam: 28 Jun 2026 · Closed book · Every slide example worked in full

How to use this companion. Read the [blue intuition boxes](#) first — they give the idea in plain words. [Amber everyday boxes](#) give a real-life analogy. [Green worked boxes](#) show a problem solved step by step. [Red watch-out boxes](#) name the most common mistakes. [Purple key-takeaway boxes](#) give the single line to remember.

1 What is AI?

Artificial Intelligence is the study of rational agents. A **rational agent** is anything that perceives its environment and acts to maximise a performance measure. The performance measure is your scoring rule — it says what counts as success.

The intuition

An agent is not just a robot. It is any decision-maker: a chess program, a spam filter, a thermostat. What makes it *rational* is that it picks the action most likely to score well, given everything it currently knows.

★ Everyday picture

Think of a taxi driver. The environment is the city. The sensors are eyes and GPS. The actuators are the steering wheel and pedals. The performance measure is: arrive safely, on time, and cheaply. The driver (agent) picks routes that maximise those goals. An AI taxi agent does the same — but in silicon.

The agent–environment loop works like this. At each time step, the agent receives a **percept** (a reading from its sensors). It processes that percept using its internal mechanism. Then it selects and executes an **action**. The environment changes. A new percept arrives. The cycle repeats.

Where this shows up in ML. Reinforcement learning is the direct formalisation of this loop. The agent is the policy network; the environment is the simulator; the percept is the observation vector; the action is the output token or motor command; the performance measure is the cumulative reward.

✓ Key takeaway

AI = study of rational agents. A rational agent picks the action expected to maximise its performance measure given its percepts so far.

2 PEAS Specification

Before building an agent, we describe its task with four components. The acronym is **PEAS**: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

 The intuition

PEAS is a job description for the agent. Performance says what success looks like. Environment says where the agent lives. Actuators say what it can do. Sensors say what it can see.

 Worked example: Vacuum cleaner PEAS

Step 1. Performance measure. Maximise cleanliness of all squares. Minimise energy used. Minimise time.

Step 2. Environment. A grid of squares, each either dirty or clean. The agent starts at some square.

Step 3. Actuators. Move Left, Move Right, Suck (clean the current square).

Step 4. Sensors. Location sensor (which square am I on?). Dirt sensor (is this square dirty or clean?).

Full PEAS: Performance = clean squares / energy / time; Environment = 2-square grid; Actuators = Left, Right, Suck; Sensors = location, dirt-detector.

 Worked example: Self-driving car PEAS

Step 1. Performance. Safe arrival, minimal travel time, legal driving, passenger comfort.

Step 2. Environment. Roads, other vehicles, pedestrians, weather, traffic signals.

Step 3. Actuators. Steering wheel, accelerator, brake, horn, turn signals.

Step 4. Sensors. Cameras, LIDAR, GPS, speedometer, odometer, microphones.

 Worked example: Sliding-tile adversarial game (exam sample question)

The exam gives a board with 3 A-tiles, 3 B-tiles, and 2 empty spaces. Players alternate sliding or hopping tiles. The goal is 3 identical tiles adjacent.

Step 1. Performance. Win the game (get 3 similar tiles adjacent first).

Step 2. Environment. The board — a grid with A-tiles, B-tiles, and empty spaces.

Step 3. Actuators. Slide an adjacent tile into an empty space; hop one tile over another into an empty space.

Step 4. Sensors. Full view of the current board configuration (fully observable).

Where this shows up in ML. Every RL problem starts with a PEAS-like specification. In OpenAI Gym, the environment defines the state space (sensors), action space (actuators), and reward function (performance). The “E” in PEAS maps directly to the `env` object.

 Key takeaway

PEAS = Performance, Environment, Actuators, Sensors. Write all four before designing any agent.

3 Environment Properties — 6 Pairs

Every environment is described along six independent dimensions. For exam questions, identify the property and *justify briefly*.

 The intuition

These six pairs are a checklist. You read the problem description, tick each pair, and you have a complete picture of how hard the agent’s task will be.

Property	One side	Other side
Observability	Fully observable	Partially observable
Agents	Single agent	Multi-agent (competitive / cooperative)
Determinism	Deterministic	Stochastic
Episodicity	Episodic	Sequential
Dynamics	Static	Dynamic
State space	Discrete	Continuous

Fully observable means the agent's sensors give it the complete state at every step (chess board). **Partially observable** means some state is hidden (poker — you can't see opponents' hands).

Deterministic means the next state is completely determined by the current state and the action taken. **Stochastic** means there is uncertainty (rolling dice, slippery floors).

Episodic means each task is independent — what happened in episode 1 has no effect on episode 2 (sorting mail: each email is separate). **Sequential** means past actions affect future decisions (chess: every move changes what's possible).

Static means the environment doesn't change while the agent is thinking (crossword puzzle). **Dynamic** means it can change at any time (driving — other cars keep moving).

Worked example: Classify: online chess against a human

Step 1. Fully observable — both players see the entire board.

Step 2. Multi-agent (competitive) — two players, each trying to win.

Step 3. Deterministic — pieces move to exactly the stated square, no dice.

Step 4. Sequential — every past move restricts future legal moves.

Step 5. Static — the board does not change while you think (no time pressure changes state).

Step 6. Discrete — finite number of squares and piece types.

Result: Fully observable, Multi-agent competitive, Deterministic, Sequential, Static, Discrete.

Watch out

Do not confuse *static* with *deterministic*. A static environment simply does not change while the agent deliberates. A deterministic environment has no randomness in the outcome of actions. An environment can be deterministic and dynamic at the same time (e.g. a fast-moving but physics-exact simulation).

Also: *multi-agent* doesn't always mean *competitive*. Two robots co-operating to move furniture are multi-agent but cooperative.

Where this shows up in ML. RL environments in research are often described by exactly these properties. Atari games are fully observable, single-agent, deterministic, episodic, dynamic, discrete. Real-world robotics is partially observable, single-agent, stochastic, sequential, dynamic, continuous — much harder.

Key takeaway

Six pairs: (1) observability, (2) number of agents, (3) determinism, (4) episodic vs sequential, (5) static vs dynamic, (6) discrete vs continuous. Memorise all six; the exam asks you to justify each one.

4 The 5 Agent Types

Agents are classified by how sophisticated their decision-making is.

The intuition

Each type adds one more layer of intelligence. Simple reflex just reacts. Model-based remembers the world. Goal-based plans ahead. Utility-based trades off competing goals. Learning agents improve over time.

1. **Simple Reflex Agent.** Condition–action rules only. If dirty, suck. If at edge, turn. No memory of the past. Fails in partially observable environments.
2. **Model-Based Reflex Agent.** Keeps an internal model of the world to handle partial observability. The model tracks what it cannot directly see.
3. **Goal-Based Agent.** Has an explicit goal. Searches for sequences of actions that reach the goal. More flexible than reflex — can plan.
4. **Utility-Based Agent.** Has a utility function mapping states to real numbers. Chooses actions that maximise expected utility. Handles competing goals (fast *and* safe).
5. **Learning Agent.** Starts with limited knowledge and improves through experience. Has four components: learning element (updates), performance element (acts), critic (evaluates), problem generator (experiments).

Watch out

The exam often asks: “Which agent type is best for environment X?” A simple reflex agent cannot handle partial observability. A goal-based agent cannot handle competing goals elegantly. Use a utility-based or learning agent for real-world complexity.

Where this shows up in ML. Deep RL agents are learning agents: the neural network is the performance element, gradient descent is the learning element, the reward is the critic, and exploration strategies (epsilon-greedy, UCB) are the problem generator.

Key takeaway

5 types: Simple reflex → Model-based reflex → Goal-based → Utility-based → Learning agent. Each type is strictly more powerful than the one before it.

5 Problem-Solving Agents

A problem-solving agent formulates a search problem and finds a solution. It has four steps: **Goal formulation** (what do we want?), **Problem formulation** (what states and actions?), **Search** (find a sequence of actions), **Execution** (carry it out).

A formal search problem has five components:

- **State space** — all possible configurations.
- **Initial state** — where we start.
- **Actions** — what we can do.
- **Transition model** — $\text{Result}(s, a) \rightarrow s'$.

- **Goal test** — is this state the destination?
- **Path cost** — numeric cost of a path (sum of step costs).

A **solution** is a sequence of actions leading from the initial state to a goal state. An **optimal solution** minimises path cost.

Where this shows up in ML. Hyperparameter search, architecture search (NAS), and planning in model-based RL all reduce to search problems. The state space is the space of hyperparameter combinations or network topologies.

✓ **Key takeaway**

Problem-solving agent = goal formulation + problem formulation + search + execute. A solution is any path to the goal; an optimal solution is the cheapest such path.